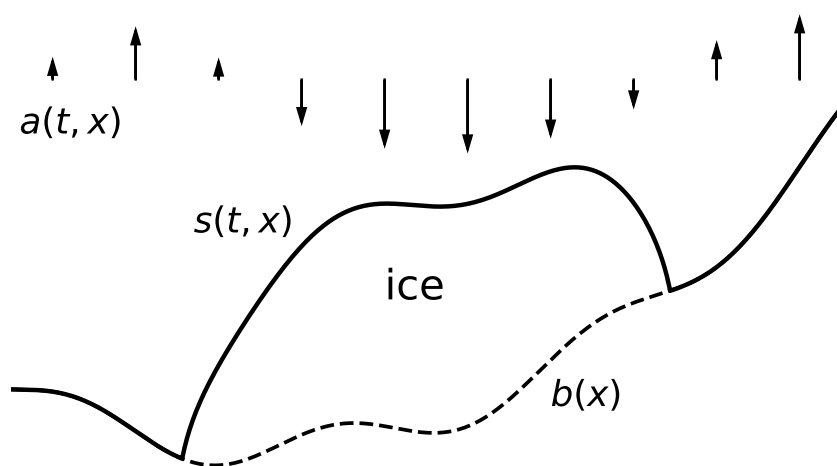


NUMERICAL MODELING OF GLACIERS

Ed Bueler

International Summer School in Glaciology
McCarthy, Alaska
2010–2026



Some notation used in these notes.

1. INTRODUCTION

Purpose. The greatest importance of mathematical and numerical models to science, and glaciology in particular, is *introspection*. In building and using a model you are asking yourself: when I combine my imperfect and incomplete understanding of glacier processes into this model, does it behave as I expect? A numerical simulation only reports the consequences of the equations, inadequacies and all, but the approximation can show how processes interact to give overall behavior, and it can sometimes demonstrate flaws in your understanding of those processes. While all computer programs produce numbers, we want numbers that closely approximate our intended continuum equations—therefore the differential equations, not just the computer programs, will be a leading focus.

Scope. These notes are deliberately limited in scope: *surface kinematics and shallow approximations of ice flow*. (Defining these technical terms is a big part of our agenda.) Numerical modeling will be approached through *understandable and short Python codes that do particular jobs*.

Download and Python codes. These notes, some slides, additional exercises, and the Python programs are all distributed by cloning the `git` repository at

<https://github.com/bueler/mccarthy>

The programs are in the `py/` subdirectory.

Notation. Our notation, which generally follows references [12, 26], is in Table 1. Cartesian coordinates are x, y, z and time t , with z positive-upward. Tensor notation uses subscripts from $\{1, 2, 3, i, j\}$, so τ_{ij} and τ_{13} denote entries of the deviatoric stress tensor.

With this notation in hand, the next Section introduces the standard equations for ice flow, the Glen-Stokes model, to motivate later approximations.

2. ICE FLOW EQUATIONS

Glacier ice is a weird fluid. Ice in glaciers behaves as a viscous fluid [12], and we describe its motion by a velocity field $\mathbf{u}$. It is incompressible. If the ice were faster-moving than it is in reality, and linearly-viscous like liquid water, then we would use the Navier-Stokes partial differential equations as the model for this typical fluid [28]:

$$\nabla \cdot \mathbf{u} = 0 \qquad \text{incompressibility} \qquad (1)$$

$$\rho(\mathbf{u}_t + \mathbf{u} \cdot \nabla \mathbf{u}) = \nabla \cdot (\nu \nabla \mathbf{u}) - \nabla p + \rho \mathbf{g} \qquad \text{stress balance} \qquad (2)$$

Besides the velocity $\mathbf{u}$, the other symbols in these equations are the ice density ρ , the viscosity ν , the pressure p , and the acceleration of gravity $\mathbf{g}$. The velocity field $\mathbf{u}$ is divergence-free, equation (1), thus no fluid is created or lost within the domain, which is one aspect of mass conservation. Equation (2) is a form of “ $ma = F$ ”, Newton’s second law. The left side of (2) has the acceleration following the fluid, $\mathbf{u}_t + \mathbf{u} \cdot \nabla \mathbf{u}$, while the right-hand side is the total force per unit volume, the sum of viscous, pressure, and gravity ($\rho \mathbf{g} = \langle 0, 0, -\rho g \rangle$) forces, respectively. Using σ to denote the Cauchy stress tensor, the right side can be written $\nabla \cdot \sigma + \rho \mathbf{g}$.

The space of solutions to (1) and (2) is rich. It includes turbulent flows, which we will not need here. However, reading a book-length introduction to fluids such as [1], or their

TABLE 1. Notation used in these notes.

variable	description	SI units	value
a	surface mass balance	m s^{-1}	
A	ice softness in Glen law	$\text{Pa}^{-n} \text{s}^{-1}$	
b	bed elevation of the glacier	m	
$D\mathbf{u}$	strain rate tensor	s^{-1}	
D^{SIA}	diffusivity in shallow ice approximation	$\text{m}^2 \text{s}^{-1}$	
∇	(spatial) gradient	m^{-1}	
$\nabla \cdot$	(spatial) divergence	m^{-1}	
$\mathbf{g}$	acceleration of gravity; $g = \mathbf{g} $	m s^{-2}	9.81
H	ice thickness: $H = s - b$	m	
n	exponent in Glen flow law		3
$\mathbf{n}_s$	normal vector: $\mathbf{n}_s = \left\langle -\frac{\partial s}{\partial x}, -\frac{\partial s}{\partial y}, 1 \right\rangle$		
ν	viscosity or effective viscosity	Pa s	
p	pressure	Pa	
$\mathbf{q}$	map-plane ice flux: $\mathbf{q} = \int_b^s \mathbf{U} dz$	$\text{m}^2 \text{s}^{-1}$	
ρ	density of ice	kg m^{-3}	910
s	surface elevation of the glacier	m	
σ	Cauchy stress tensor: $\sigma_{ij} = \tau_{ij} - p \delta_{ij}$	Pa	
τ	deviatoric stress tensor	Pa	
$\mathbf{u}$	$\mathbf{u} = \langle u_1, u_2, u_3 \rangle = \langle u, v, w \rangle$ 3D ice velocity	m s^{-1}	
$\mathbf{U}$	$\mathbf{U} = \langle u_1, u_2 \rangle = \langle u, v \rangle$ horizontal ice velocity	m s^{-1}	

numerical solution (*computational fluid dynamics*; e.g. [28]), may not even be helpful for glacier modeling because ice is not actually modeled by equation (2).

One might ask: is glacier modeling really computational fluid dynamics at all? The answer is *yes*. Glacier ice flow is a large-scale, climate-relevant fluid problem, comparable to ocean circulation, for example. However, phenomena which make ocean flows exciting, including turbulence, convection, Coriolis force, and density stratification, are all absent. What could be interesting about the slow flow of ice? How does it work, anyway?

Glen-Stokes equations. We have said that glacier ice is a “slow” fluid. In terms of equation (2), this means that $\rho(\mathbf{u}_t + \mathbf{u} \cdot \nabla \mathbf{u}) \approx 0$, the forces (stresses) of inertia are negligible, so Newton’s second law becomes a *stress balance*. Ice is also a shear-thinning fluid, with *non-Newtonian* behavior; larger strain rates imply smaller viscosity. The nonlinear viscosity ν is thus not constant, and an empirically-based flow law is needed. Combining these few ideas gives the canonical ice flow model:

$$\nabla \cdot \mathbf{u} = 0 \quad \text{incompressibility} \quad (3)$$

$$-\nabla \cdot \tau + \nabla p = \rho \mathbf{g} \quad \text{stress balance} \quad (4)$$

$$D\mathbf{u} = A|\tau|^{n-1}\tau \quad \text{Glen-Steinmann-Nye flow law} \quad (5)$$

Here $\rho > 0$ and $\mathbf{g}$ are constant, and the *ice softness* $A > 0$ is constant for our simplified treatment, and n is often taken to be 3. The other fields depend on t, x, y, z .

Equations (3)–(5) are commonly called the *Stokes model* in glaciology. Regarding the details, note that $|\tau|^2 = (1/2)\tau_{ij}\tau_{ij}$, using the summation convention, which defines the *effective deviatoric stress* $|\tau|$. By definition $D\mathbf{u} = \frac{1}{2}(\nabla\mathbf{u} + \nabla\mathbf{u}^\top)$, thus $(D\mathbf{u})_{ij} = \frac{1}{2}((u_i)_{x_j} + (u_j)_{x_i})$ in components. In (5) the deviatoric stress τ and strain rate $D\mathbf{u}$ tensors are proportional, with a scalar power-law coefficient $A|\tau|^{n-1}$; this was Nye’s contribution to the flow law form. Since $D\mathbf{u}$ is symmetric, it follows from (3) and (5) that both tensors τ , $D\mathbf{u}$ are symmetric and have trace zero.

There are many concepts about the Stokes model which one might learn. For example, the many possible solutions of (4)–(5) are constrained by incompressibility (3), and the pressure p serves as a Lagrange multiplier for this constraint.

However, note something simple: equations (3)–(5) do not contain any time derivatives! This implies that the combined effect of glacier geometry, boundary stresses, and gravity will determine [18] the solution velocity and stress fields $(\mathbf{u}, p, \tau)$ *instantaneously*. Velocity is therefore a *diagnostic* variable: ice flow codes need no stored velocity to restart, unlike in a Navier-Stokes simulation.

The Glen-Stokes equations apply instantaneously at every time, but as the glacier interacts with the climate its geometry and basal stresses will change, so the $\mathbf{u}$ and p solution fields will also change. In other words, though equations (3)–(5) have no time derivatives, the glacier they model is *not* (generally) in steady state.

Planar Stokes model. Consider the x, z -coordinates *planar* (or *flow line*) and $n = 3$ case. The out-of-plane velocity component in equations (3)–(5) is zero, i.e. $v = 0$, and derivatives with respect to y are also zero:

$$u_x + w_z = 0 \quad \text{incompressibility} \quad (6)$$

$$-\tau_{11,x} - \tau_{13,z} + p_x = 0 \quad \text{stress balance } (x) \quad (7)$$

$$-\tau_{13,x} + \tau_{11,z} + p_z = -\rho g \quad \text{stress balance } (z) \quad (8)$$

$$u_x = A\tau^2\tau_{11} \quad \text{flow law (diagonal)} \quad (9)$$

$$u_x + w_x = 2A\tau^2\tau_{13} \quad \text{flow law (off-diagonal)} \quad (10)$$

Here τ_{13} is a shear stress, while τ_{11}, τ_{33} are deviatoric longitudinal stresses. In this planar case it follows that $\tau_{33} = -\tau_{11}$, by incompressibility (6) and (9)–(10), which explains why we can write (8) without τ_{33} . Also, a short calculation shows $\tau^2 = \tau_{11}^2 + \tau_{13}^2$.

Observe that equations (6)–(10) form a system of 5 nonlinear equations in 5 unknown scalar fields $(u, w, p, \tau_{11}, \tau_{13})$. Note that a Firedrake-based [24] Stokes solver is in `py/stokes/`, documented by a technical appendix there, which solves this model.

Slab-on-a-slope solution. However, these 5 equations are still complicated and difficult to understand! Before jumping into numerical solution methods we consider a simplified situation, a uniform slab of ice on an incline. Here we can exactly solve the Stokes equations. The same calculation motivates a later shallow model.

We rotate our coordinates only for this calculation. The x, z axes shown in Figure 1 are rotated clockwise at angle $\alpha > 0$ so that the gravity vector now has components

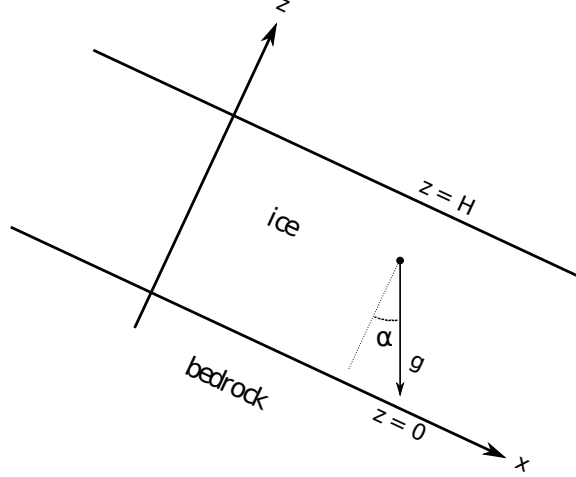


FIGURE 1. Rotated axes for the slab-on-a-slope calculation.

$\mathbf{g} = (g \sin \alpha, -g \cos \alpha)$. Equations (7) and (8) have the new form

$$-\tau_{11,x} - \tau_{13,z} + p_x = \rho g \sin \alpha, \quad (11)$$

$$-\tau_{13,x} + \tau_{11,z} + p_z = -\rho g \cos \alpha. \quad (12)$$

For this uniform slab we assume further that there is no variation with x . Then the system of 5 equations simplifies greatly:

$$w_z = 0, \quad \tau_{11} = 0, \quad u_z = 2A\tau^2\tau_{13}, \quad \tau_{13,z} = -\rho g \sin \alpha, \quad p_z = -\rho g \cos \alpha. \quad (13)$$

Each unknown is now a function of z only. Boundary conditions come from assuming that the ice is not crossing the bed and that the stress-free ice surface is moving parallel to the x -axis. Thus: $w(0) = 0$, $p(H) = 0$, $\tau_{13}(H) = 0$. We already have $\tau_{11}(z) = 0$, and by integrating equations from (13) with respect to z , we get $w(z) = 0$, $p(z) = (\rho g \cos \alpha)(H - z)$, and $\tau_{13}(z) = (\rho g \sin \alpha)(H - z)$. Note that $H - z$ is the depth below the ice surface; pressure and shear stress are proportional to depth. Integrating the remaining equation yields the horizontal velocity function, using notation $u(0) = u_0$:

$$u(z) = u_0 + \frac{1}{2}A(\rho g \sin \alpha)^3 (H^4 - (H - z)^4) \quad (14)$$

Do we believe formula (14), which makes a specific prediction about the profile of flow in a glacier? Figure 2 compares observations of a mountain glacier, showing that this model does a credible job of capturing deformation flow in this case.

However, the basal velocity $u_0 = u(0)$ is generally unknown. For simplicity we will assume in these notes that there is no sliding ($u_0 = 0$), for example because the ice is frozen to the bed. (Effective models for glacier sliding are in the literature, e.g. [7].) In Section 4 we will convert the severe simplifications of this slab-on-a-slope model into the *shallow ice approximation*.

Forms of the flow law. Returning to the general case, flow law (5) has another important form. Let us consider the $n = 3$ case for simplicity. Recall that $|\tau|^2 = (1/2)\tau_{ij}\tau_{ij}$, and define $|D\mathbf{u}|^2 = (1/2)(D\mathbf{u})_{ij}(D\mathbf{u})_{ij}$; the scalars $|\tau|$ and $|D\mathbf{u}|$ are tensor *norms*. From

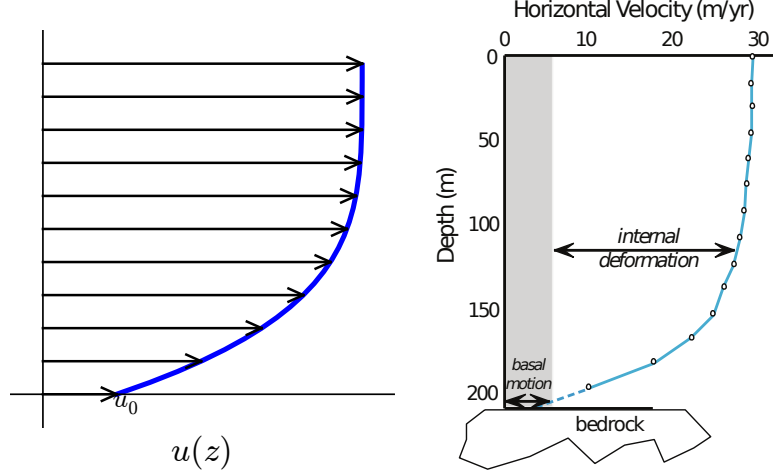


FIGURE 2. Left: Velocity from formula (14). Right: Measured velocity in a glacier (Athabasca Glacier [25]).

(5) we get $|D\mathbf{u}| = A|\tau|^3$. Thus $|\tau| = A^{-1/3}|D\mathbf{u}|^{1/3}$, and (5) can be rewritten

$$\tau = 2\nu D\mathbf{u} \quad \text{viscosity form of a flow law} \quad (15)$$

where the (nonlinear) *effective viscosity* ν has this form:

$$\nu = \frac{1}{2}A^{-1/3}|D\mathbf{u}|^{-2/3}. \quad (16)$$

Equations (15) and (16) eliminate the stress tensor τ from the stress balance (4), so that the Stokes formulas only need velocity $\mathbf{u}$, pressure p , and strain rates $D\mathbf{u}$ as major variables. Often $B = A^{-1/3}$ is called the *ice hardness*.

Hydrostatic approximations. A relatively-mild simplification of the Glen-Stokes equations comes from assuming that horizontal variation in the vertical shear stress is small compared to the other terms. This removes the single term $\tau_{13,x}$ from the z -component of the (planar) stress balance (8), giving

$$\tau_{11,z} + p_z = -\rho g. \quad (17)$$

Because $\sigma_{ij} = \tau_{ij} - p\delta_{ij}$, so that $\tau_{11} + p = -\tau_{33} + p = -\sigma_{33}$, equation (17) says that the vertical normal stress σ_{33} is linear in depth; it is *hydrostatic*. (Note that the pressure p is not generally hydrostatic in this model.) From a surface value of $\sigma_{33} = 0$ we get

$$\tau_{11} + p = \rho g(s - z), \quad (18)$$

where $z = s(t, x)$ is the *surface elevation*. Equation (18) allows us to eliminate p from the model equations (6)–(10). Taking the x -derivative of (18), substituting into (7) to eliminate p_x , and using the viscosity form (15) leads to this equation [12]:

$$(4\nu u_x)_x + (\nu(u_z + w_x))_z = \rho g s_x \quad \text{hydrostatic stress balance} \quad (19)$$

Equations (6), (16), (19), plus appropriate boundary conditions, determine u and w , without the need to simultaneously solve for p . (The remaining unknowns can be determined by differentiation and/or integration in z .) However, while p is gone, (19) remains non-trivially coupled to incompressibility (6) through the vertical velocity w .

We get the even-simpler Blatter-Pattyn model [23], sometimes called the *first-order approximation* [12], by dropping w_x from (19):

$$(4\nu u_x)_x + (\nu u_z)_z = \rho g s_x \quad \text{Blatter-Pattyn stress balance} \quad (20)$$

Now solving (16), (20) as a boundary value problem yields the scalar horizontal velocity u . We can then recover w using incompressibility (6) and integration in z . Because it has a form like the Poisson equation, the process of solving equation (20) was more familiar to practitioners than (19), which is one way to explain its significance.

3. SURFACE KINEMATICAL EQUATION, AND THE EVOLUTION OF GLACIERS

Time-dependent ice domain. The Glen-Stokes stress balance model for ice flow, is incomplete as a model for glacier behavior in a very important sense. It describes the ice flow at every instant, but it gives no direct clue about how glaciers *change*. The stress balance equations contain no time derivatives, and they can not directly “see” the climate around the glacier,¹ other than gravity.

On the other hand, the *domain* on which the stress balance equations are solved evolves in time. We need notation for this time-dependent geometry. Let $s(t, x, y)$ be the ice surface elevation in 3D; the surface itself is $z = s(t, x, y)$. We assume here that the glacier bed elevation $b(x, y)$ is time-independent, but erosion or earth deformation (e.g. glacial isostasy) would make the bed elevation time-dependent, in which case *basal kinematical equation* (33) would also be needed. The stress balance differential equations, whichever Section 2 model is chosen, are then solved on the 3D set

$$\{(x, y, z) : b(x, y) < z < s(t, x, y)\}. \quad (21)$$

This is the changing set of 3D points between the surface and the bed. In glacier equations it is primarily the ice domain (21) that evolves in time. The stress balance equations are instantaneous truths which hold on this evolving domain. The Figure on the cover shows this notation in the planar (2D) case, including $z = s(t, x)$ and $z = b(x)$, and the 2D domain $\{(x, z) : b(x) < z < s(t, x)\}$ is labeled ice.

Surface mass rates. The upper surface evolves in part through the *surface mass balance* (SMB) rate, also known as the *climatic* mass balance [9], denoted by $a(t, x, y)$. (“SMB” is one of a few abbreviations used in these notes; see Table 2.) Our elevations are in meters, and time in seconds, so a is measured in ice-equivalent meters per second.

By definition, the SMB is the balance between accumulating snow and the loss of water, by melt and runoff of the snow, firn, or ice, and by sublimation. Here and in most glacier literature, the speed of liquid runoff is assumed to be so much larger than the ice flow speed that the runoff acts instantaneously. Also, we only consider simulations of land-based glaciers and we omit the thermodynamically-dominated *basal melt*, a component of the *basal mass balance* rate, which can be of either sign because of *freeze-on*.

¹A the stress balance “sees” basal stress as a boundary condition, and this is sometimes regarded as a climate input, one which acts via highly-nontrivial additional models.

TABLE 2. Abbreviations, listed alphabetically.

abbreviation	meaning	definition/citations
CFL	numerical stability criterion	inequality (29); [10, 19, 21]
FD	finite difference (method)	[19, 21]
PDE	partial differential equation	
SIA	shallow ice approximation	equations (39), (43); [12]
SKE	surface kinematical equation	equation (24); [12]
SMB	surface mass balance	[9]

Surface kinematics. The upper surface evolves according to the *surface kinematical equation* (SKE; [26]), which in the planar case is the equation

$$\frac{\partial s}{\partial t} = a + w - u \frac{\partial s}{\partial x}. \quad (22)$$

Regarding the notation, the planar ice velocity is $\mathbf{u} = \langle u, w \rangle$ in components, but equation (22) uses the component values at the upper surface $z = s(t, x)$, i.e. $u = u(t, x, s(t, x))$ and $w = w(t, x, s(t, x))$.

Equation (22) says that the ice surface moves up and down, at rate $\partial s / \partial t$, according to three contributions, the SMB a , the vertical ice velocity w , and the product of horizontal ice velocity u with the surface slope $\partial s / \partial x$. Dimensionally, at least, this is correct; $\partial s / \partial x$ is unitless, and all terms in (22) are in meters per second.

Vector notation is convenient when extending (22) to 3D. Here the surface elevation function $s(t, x, y)$ depends on two horizontal variables, and we define

$$\mathbf{n}_s = \left\langle -\frac{\partial s}{\partial x}, -\frac{\partial s}{\partial y}, 1 \right\rangle, \quad (23)$$

an upward-pointing normal vector on the surface. Then the SKE is

$$\frac{\partial s}{\partial t} - \mathbf{u} \cdot \mathbf{n}_s = a \quad (24)$$

where $\mathbf{u} = \langle u, v, w \rangle$ is evaluated at the surface.

The SKE is described in the glacier literature as a mere boundary condition [12, 26], but it is actually the dominant statement of mass conservation in a glacier model. It can be derived from conservation by a standard “pillbox” calculus argument [2]. Equation (24) suggests we might think of the climate a as given data, and we might try to solve it for s , which is exactly what we are about to do.

First numerical glacier model. Regarding equation (24) one might say:

by numerically solving this equation for s , one has a model for a glacier.

Perhaps it is better to say that the SKE is *correct* in a physical sense and can be *understood* as a minimal glacier model.

The plan is to numerically solve the planar SKE (22) for the surface elevation $s(t, x)$, based on the bold assumption that the SMB function $a(t, x)$, and the velocity surface components $u(t, x)$ and $w(t, x)$, are given by other agents. In practice the SMB could be the output of a mass/energy process sub-model, coupled to atmosphere physics and observations [27]. The surface velocity components u, w would likewise be the output

of a stress balance sub-model (Sections 2 and 4). However, the stress balance depends strongly on the surface elevation s , which determines the domain geometry, and so in fact the whole problem is non-trivially *coupled*.

But we must start somewhere. This first numerical model is in a short Python program `py/surface.py`. (See “Download” in Section 1.) It generates a finite difference approximation to the surface elevation $s(t, x)$, over intervals of space $0 < x < L$ and time $0 < t < T$, according to (22), by an *explicit* time-dependent numerical method. We will use this first program to consider numerical stability and other practical issues.

Finite differences. Though textbooks [19, 21] are recommended, *finite difference* (FD) approximations need an introduction here. From calculus, Taylor’s theorem says that for a smooth function $f(x)$ and a real number Δ , we have an expansion:

$$f(x + \Delta) = f(x) + f'(x)\Delta + \frac{1}{2}f''(x)\Delta^2 + \frac{1}{3!}f'''(x)\Delta^3 + \dots \quad (25)$$

(The precise theorem uses a finite sum and a remainder formula, but a loose version will suffice here.) One may replace “ Δ ” by its negative, or other multiples, to get related forms such as $f(x - \Delta) = f(x) - f'(x)\Delta + \frac{1}{2}f''(x)\Delta^2 - \dots$

FD numerical models are constructed by combining expressions like (25) to approximate the derivatives in the continuum equations. The resulting algebraic equations relate unknown gridded values via mere arithmetic, which digital computers can do well. As a simple example, subtract $f(x)$ from both sides of (25), then divide by Δ , to find an approximation of the first derivative,

$$\frac{f(x + \Delta) - f(x)}{\Delta} \approx f'(x).$$

This is usually written to emphasize the size of the remaining terms in the expansion, i.e. the size of the remainder in Taylor’s theorem:

$$f'(x) = \frac{f(x + \Delta) - f(x)}{\Delta} + O(\Delta).$$

Here $O(\Delta)$ denotes a quantity with magnitude smaller than a fixed multiple of Δ , asymptotically as $\Delta \rightarrow 0$.

To apply this FD approach to a *partial* differential equation (PDE) like SKE (22), let us choose a time step $\Delta t > 0$ and a grid spacing $\Delta x > 0$. Apply Taylor expansions in each variable to approximate the partial derivatives in the equation:

$$\begin{aligned} \frac{\partial s}{\partial t}(t, x) &= \frac{s(t + \Delta t, x) - s(t, x)}{\Delta t} + O(\Delta t), \\ \frac{\partial s}{\partial x}(t, x) &= \frac{s(t, x + \Delta x) - s(t, x - \Delta x)}{2\Delta x} + O((\Delta x)^2). \end{aligned} \quad (26)$$

The first formula is a *forward difference* and the second a *centered difference*.

Our numerical solution will use a spatial grid $\{x_j\}$, with equal spacing Δx and an integer index j . It will use a temporal grid $\{t_n\}$ with (possibly unequal) spacing Δt . We seek a new surface elevation $s(t_{n+1}, x_j)$ from some current values $s(t_n, x_\bullet)$. However, we should distinguish between $s(t_n, x_j)$, a value of the continuum exact solution at a grid point, and our numerical approximation of it at the same point, now denoted s_j^n . Conceptually,

$s_j^n \approx s(t_n, x_j)$, but the values s_j^n are the ones we actually get by computation. Finally, we will declutter the grid notation by writing s_j for s_j^n and s_j^{new} for s_j^{n+1} .

With these conventions, the FD approximations (26) convert the SKE (22) into a discrete, algebraic equation:

$$\frac{s_j^{\text{new}} - s_j}{\Delta t} = a_j + w_j - u_j \frac{s_{j+1} - s_{j-1}}{2\Delta x}.$$

This will be a line of code which determines s_j^{new} , so we rewrite it that way:

$$s_j^{\text{new}} = s_j - \Delta t u_j \frac{s_{j+1} - s_{j-1}}{2\Delta x} + \Delta t(a_j + w_j). \quad (27)$$

Note that this scheme uses 3 values at the current time to determine one value at the new time. A *stencil diagram* captures this meaning (Figure 3, left). The standard name for scheme (27) is *forward-time, centered-space*.

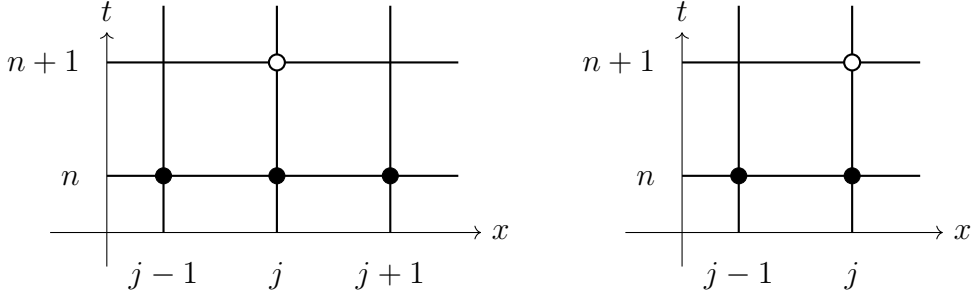


FIGURE 3. To compute $s_j^{\text{new}} = s_j^{n+1}$ (circle), scheme (27) (left) uses 3 gridded values at the current time t_n (solid), while the upwind scheme (28) (right; $u_j > 0$ case) uses just 2, depending on the flow direction.

However, if we run a simulation based on scheme (27) then we will see wiggles which do not go away. The possibly-surprising remedy is to revert to a less-accurate and asymmetric FD choice for the spatial derivative [21]. For $\partial s / \partial x$ we take

$$\frac{\partial s}{\partial x}(t, x) = \frac{s(t, x) - s(t, x - \Delta x)}{\Delta x} + O(\Delta x)$$

at grid points (t_n, x_j) where $u_j > 0$, and otherwise ($u_j \leq 0$)

$$\frac{\partial s}{\partial x}(t, x) = \frac{s(t, x + \Delta x) - s(t, x)}{\Delta x} + O(\Delta x).$$

This is called *upwind* differencing. These formulas have only first-order accuracy, versus the centered-difference in (26), but they permit (conditional) stability and the avoidance of wiggles. Written like (27), the upwind scheme is

$$s_j^{\text{new}} = s_j - \Delta t \frac{u_j}{\Delta x} \begin{cases} s_j - s_{j-1}, & u_j > 0 \\ s_{j+1} - s_j, & u_j \leq 0 \end{cases} + \Delta t(a_j + w_j). \quad (28)$$

The stencil in Figure 3 right shows the $u_j > 0$ case in which the velocity feeds the ice from the left, which explains the “upwind” naming.

Time-stepping implementation. We can now look at the code. Upwind scheme (28) for SKE (22) is a short Python function in the program `py/surface.py`:

```
def explicitupwindstep(x, s, dt):
    dx = x[1] - x[0]
    snew = s.copy()
    snew[1:] -= dt * u(x[1:]) * (s[1:] - s[:-1]) / dx
    snew[1:] += dt * (a(x[1:]) + w(x[1:]))
    return snew
```

In detail, `x`, `s`, and `snew` are one-dimensional NumPy [15] arrays, and `x` is assumed to hold an equally-spaced grid. Separate “data” functions must define $a(x)$, $u(x)$, and $w(x)$. In this implementation we need the formula for $u(x)$ to be positive, but the two cases in (28) would be easily handled by a conditional statement. That `snew[0] = s[0]` never changes is actually a *boundary condition* compatible with the fact that the velocity is inward at the left end ($u(x[0]) > 0$). Also note that the left end of our example glacier, see below, has zero thickness, that is, `s[0] = b(0)`.

The rest of the code `py/surface.py` is worth a look, but there is no need to print it here. It contains formulas for the data functions, a time-stepping loop, calls to `matplotlib` to generate figures, and generous comments. Various default choices are made for convenience and illustration, for example $\Delta t = 1$ a and $\Delta x = 1$ km. Run the code as follows:

```
$ cd py/
$ python3 surface.py
evolution mode ...
    stability constant = 0.5000
    writing 50 steps to image files output/frame???.png
steady state mode ...
    writing to image file output/steady.png
```

The initial condition and final result (50th time step) are shown in Figure 4.

Stability and CFL. These first results look good (and boring). However, easy modifications reveal stability issues, exposing high-spatial-frequency, rapidly-growing oscillations. In particular, each of these separate modifications, a recommended exercise, generate such exploding wiggles:

- increase velocity: $u(x) \rightarrow 2.5 u(x)$
- increase resolution: $\Delta x = 1000 \text{ m} \rightarrow \Delta x = 400 \text{ m}$
- lengthen time-steps: $\Delta t = 1 \text{ a} \rightarrow \Delta t = 2.5 \text{ a}$

This kind of instability is well understood, and in fact the condition to avoid it has been known for 100 years. The idea is that in SKE (22) and similar *advection* equations, information travels at the speeds $\{u_j\}$. For numerical stability we need it to not travel more than spatial distance Δx during the time step Δt . Equivalently, the maximum dimensionless Courant, Friedrichs, & Lewy (CFL; [10]) ratio should not exceed one:

$$\frac{(\max_j |u_j|) \Delta t}{\Delta x} \leq 1. \quad (29)$$

The earlier destabilizing modifications each pushed this ratio over the limit.

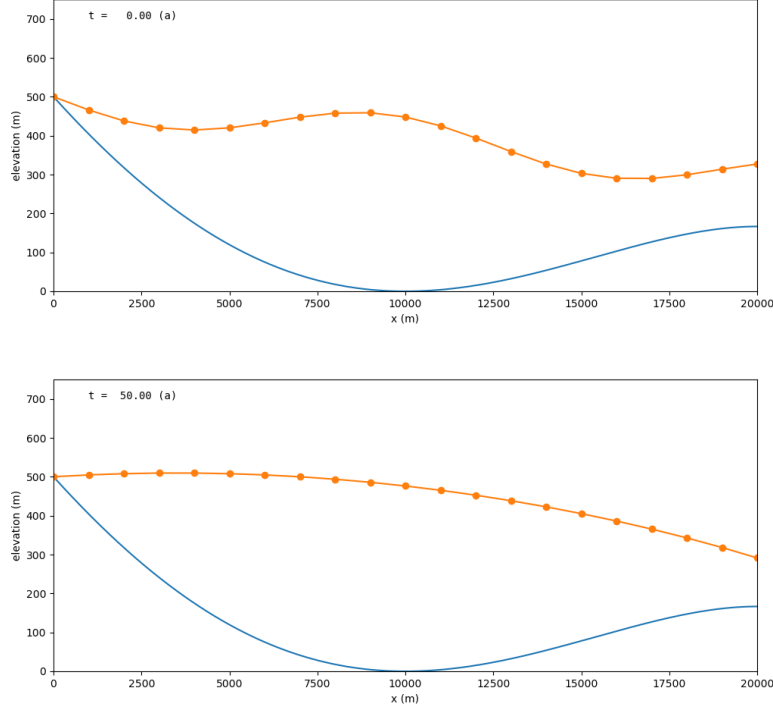


FIGURE 4. Initial condition (top) and final result (bottom) for an SKE simulation using scheme (28). The solution surface elevation is dotted.

In practice, inequality (29) is used to set the time step Δt to assure stability. Before each step the program sets $\Delta t = C \frac{\Delta x}{\max_j |u_j|}$, using $C = 0.9$ for example, so that (29) holds even when the velocities u_j are changing in time.

Admissibility. A different type of modification also breaks our method. In the default set-up, the SMB function $a(x)$ in `py/surface.py` is linear and decreasing in x , with a zero (an *equilibrium line* [9]) at $1/5$ of the total length L . This means that accumulation occurs over $0 < x < L/5$ and ablation over $L/5 < x < L$. The modification

- stronger SMB: $a(x) \rightarrow 3a(x)$

therefore strengthens the ablation over most of the domain. The final-time result is shown in Figure 5, with a glacier surface which goes below the bed. This is not good.

The bed elevation b does not even appear in equations (22) and (24), so it would seem to be irrelevant when using the SKE to evolve the glacier geometry. However, glaciers must end *where they have ablated away*, and we need to model this real effect instead of having the surface elevation beneath the bedrock. We say that the surface elevation at time t is *admissible* if

$$s(t, x, y) \geq b(x, y) \quad (30)$$

for every point x, y in the computational domain. Equivalently, this says that $H(t, x, y) \geq 0$; the thickness $H = s - b$ cannot be negative.

Our simulation using the modified SMB started from an admissible state (Figure 4, top), but it evolved to an inadmissible one (Figure 5). Avoiding this possibility, at least

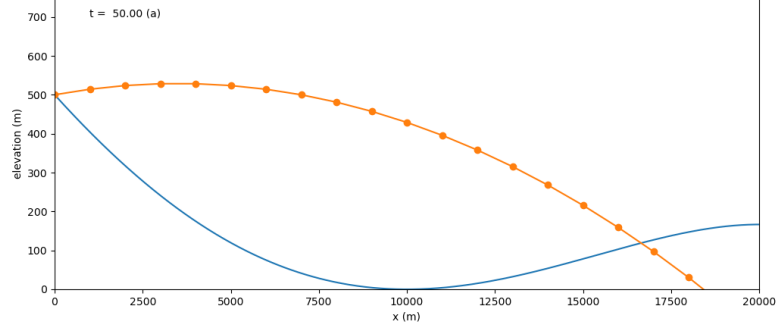


FIGURE 5. An inadmissible state, with $s < b$ in places, from stronger ablation. See inequality (30), and compare Figure 4.

in an explicit solution method like (28), is straightforward. Namely, we can *truncate* the solution, $s_j^{\text{new}} = \max\{b_j, s_j^{\text{new}}\}$ (planar case), to force admissibility.

Inequality (30), which complements definition (21) of the ice domain, and reflects the *free-boundary* nature of the glacier geometry problem [3, 4]. Bed elevation data b is thus needed even when modeling glacier surfaces, and when solving stress balances (Sections 2 and 4). Note that in unglaciated areas it makes sense to assert that $s(t, x) = b(x)$, which is compatible with how digital elevation models work in practice; the surface elevation is the top of the glacier if one is present, otherwise it is the top of the land.

Steady state. We are regarding the SKE as a minimal model which can predict glacier geometry given data for SMB and surface velocity. This approach works in *steady state* as well. Here equation (22) with $\partial s / \partial t = 0$ becomes

$$u \frac{\partial s}{\partial x} = a + w, \quad (31)$$

and (24) becomes $-\mathbf{u} \cdot \mathbf{n}_s = a$.

Using an upwind FD idea like (28), in 1D equation (31) can be solved like the time-dependent case. Specifically, because $u(x) > 0$ everywhere, our FD scheme is

$$u_j \frac{s_j - s_{j-1}}{\Delta x} = a_j + w_j \quad (\text{assumes } u_j > 0), \quad (32)$$

which is solved for s_j to give an update formula. In code, we start from the left and have a single for loop:

```
s = x.copy()
s[0] = b(0.0)
for j in range(1, J+1):
    s[j] = s[j-1] + (dx / u(x[j])) * (a(x[j]) + w(x[j]))
```

Program `py/surface.py` has solved the problem already. Viewing the output image (`output/steady.png`) shows a result very similar to Figure 4, bottom.

We have not assured admissibility in the above loop, but this is straightforward using truncation. The update formula becomes

$$s[j] = \max(b(x[j]), s[j-1] + (dx / u(x[j])) * (a(x[j]) + w(x[j])))$$

Solving the same steady-state problem in the two horizontal dimensions of a realistic glacier requires completely rethinking the solution strategy. For that we have written a very different, and more advanced, program `py/surface2d.py` which calls the Firedrake finite element library. Now the admissibility (30) inequality is integrated into the weak form of the partial differential equation (PDE) problem, as a *variational inequality* [5, Chapter 12, and references therein]. The reader is encouraged to explore this example, but we will not pursue it further in these notes.

Mass continuity equation. Equation (24), the SKE in its cleanest form, relates more-or-less observable quantities on a glacier’s surface. For example, it is a preferred equation for relating measured surface velocity to SMB. However, a different equation is often used to describe the evolution of glacier geometry, namely the *mass continuity* equation, derived next. It is essentially equivalent to (24), because of incompressibility (1), which states mass conservation within the ice.

The derivation is clearest if we state the *basal kinematical equation* first. Consider the mass balance $m(t, x, y)$ at the basal surface $z = b(t, x, y)$, the difference between melt and freeze-on. (In this derivation all quantities can depend on time.) The field m , in ice-equivalent meters per second, is positive if there is net melt. Both for grounded glaciers and floating ice shelves it can be of either sign, often through thermodynamical effects. Similar to (24), the equation is

$$\frac{\partial b}{\partial t} - \mathbf{u} \cdot \mathbf{n}_b = m. \quad (33)$$

Here $\mathbf{n}_b = \langle -\frac{\partial b}{\partial x}, -\frac{\partial b}{\partial y}, 1 \rangle$ is normal to the base and pointing upward, and “ $\mathbf{u}$ ” denotes $\mathbf{u}(t, x, y, b(t, x, y))$ on the basal surface.

If the bed elevation b is time-independent ($\partial b / \partial t = 0$) then (33) relates the ice velocity at the base to the mass balance m . For example, in the case of a grounded glacier with a flat ($\nabla b = \langle \frac{\partial b}{\partial x}, \frac{\partial b}{\partial y} \rangle = 0$) and stationary bedrock, and a melting base ($m > 0$), the vertical ice velocity at the bed surface is downward and equal to the melt: $w = -m$.

Consider the vector horizontal velocity $\mathbf{U} = \langle u, v \rangle$; compare notation for the 3D velocity $\mathbf{u} = \langle u, v, w \rangle$. The horizontal *ice flux*, also called *discharge*, is defined as the integral

$$\mathbf{q}(t, x, y) = \int_{b(t, x, y)}^{s(t, x, y)} \mathbf{U}(t, x, y, z) dz, \quad (34)$$

or simply $\mathbf{q} = \int_b^s \mathbf{U} dz$ if we suppress variables. This is a vector with two components, $\mathbf{q} = \langle q_1, q_2 \rangle$. It is often written $\mathbf{q} = \bar{\mathbf{U}}H$ where $H = s - b$ is the ice thickness, and $\bar{\mathbf{U}} = H^{-1} \int_b^s \mathbf{U} dz$ is the *vertically-averaged horizontal velocity*.

In these terms we can state the mass continuity equation itself,

$$\frac{\partial H}{\partial t} + \nabla \cdot \mathbf{q} = a - m. \quad (35)$$

This equation describes how the thickness of a moving layer of ice changes, and thus how ice volume changes. The divergence here is in horizontal variables: $\nabla \cdot \mathbf{q} = \frac{\partial q_1}{\partial x} + \frac{\partial q_2}{\partial y}$. If the bed is stationary ($\partial b / \partial t = 0$) then we can equivalently write “ $\partial s / \partial t$ ” on the left.

The net function $a - m$ on the right of (35) is the *climatic-basal mass balance* [9]. This difference of the surface and basal mass balances determines how much mass is lost or gained in a column within the glacier.

Because equation (35) looks different from the kinematical equations, we give a brief derivation. We need the *Leibniz rule* for differentiating integrals,

$$\frac{d}{dx} \left(\int_{g(x)}^{f(x)} k(x, y) dy \right) = k(x, f(x)) f'(x) - k(x, g(x)) g'(x) + \int_{g(x)}^{f(x)} \frac{\partial k}{\partial x}(x, y) dy. \quad (36)$$

Here x should be regarded as a generic independent variable, and k, f, g are generic differentiable functions. Note that “ y ” is an index (variable) of integration, and the results do not depend on its name. We apply the Leibniz rule to compute the divergence of the flux (34), paying attention to dependence on the independent variables:

$$\begin{aligned} (\nabla \cdot \mathbf{q})(t, x, y) &= \mathbf{U}(t, x, y, s(t, x, y)) \cdot \nabla s(t, x, y) - \mathbf{U}(t, x, y, b(t, x, y)) \cdot \nabla b(t, x, y) \\ &\quad + \int_{b(t, x, y)}^{s(t, x, y)} (\nabla \cdot \mathbf{U})(t, x, y, z) dz. \end{aligned}$$

The divergence $\nabla \cdot$ here is in the horizontal variables x, y , but by incompressibility (1) we can write the integrand as a vertical derivative of the vertical velocity:

$$\nabla \cdot \mathbf{U} = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = -\frac{\partial w}{\partial z}.$$

Continuing the calculation, indicating surface values by “ $\mathbf{U}|_s$ ” etc. and suppressing variables, and applying the fundamental theorem of calculus, we have

$$\nabla \cdot \mathbf{q} = \mathbf{U}|_s \cdot \nabla s - \mathbf{U}|_b \cdot \nabla b - \int_b^s \frac{\partial w}{\partial z} dz = \mathbf{U}|_s \cdot \nabla s - \mathbf{U}|_b \cdot \nabla b - w|_s + w|_b.$$

The expressions on the right are familiar from the kinematical equations (24) and (33), and in fact if we use the 3D velocity $\mathbf{u} = \langle u, v, w \rangle$, standard vector notations, and the kinematical equations then we have

$$\nabla \cdot \mathbf{q} = -\mathbf{u}|_s \cdot \mathbf{n}_s + \mathbf{u}|_b \cdot \mathbf{n}_b = a - \frac{\partial s}{\partial t} + \frac{\partial b}{\partial t} - m.$$

This says $\nabla \cdot \mathbf{q} = (a - m) - \partial H / \partial t$, which is mass continuity (35).

At this point we have three equations which might be used to describe the evolving geometry of a glacier:

- i) the surface kinematical equation (SKE) (24),
- ii) the basal kinematical equation (33), and
- iii) the mass continuity equation (35).

We derived the third using the first two and incompressibility (1). Actually, any *pair* of these three equations implies the third; one uses incompressibility and the Leibniz rule to prove the connections.

In designing a numerical glacier simulation, which equations to use for time-dependent geometry is largely a matter of taste and tradition. The most common choice in the literature is to use mass continuity (35) as the primary equation. It is also common to interpret the basal kinematical relation (33) in an ad hoc manner. Our main point here is that we can treat these equations as tools, and be free to use them.

Understanding planar mass continuity. It helps to illustrate mass continuity in the x, z (flow-line) plane. Suppose for simplicity that the base is flat ($b = 0$), so the thickness and surface elevation are the same thing ($H = s$), and suppose there is no basal mass balance ($m = 0$). In this planar case the horizontal velocity is a scalar function U , and its vertical average $\bar{U} = H^{-1} \int_0^H U dz$ and the flux $q = \int_0^H U dz = \bar{U} H$ are also scalar.

The ice flux $q = \bar{U} H$ is the rate of mass flow through the sides of a part of the glacier, for example at locations x_1 and x_2 as shown in Figure 6. Mass can be added by SMB a , distributed along the upper surface, or by a *difference* in the flux between the left and right sides. That is, the planar area A of this part of the glacier changes according to the sum of certain boundary contributions:

$$\frac{dA}{dt} = \int_{x_1}^{x_2} a dx + \bar{U}_1 H_1 - \bar{U}_2 H_2. \quad (37)$$

In three-dimensions, (37) would become an equation for dV/dt , the rate of change of the volume of a part of the glacier, which would have more sides.

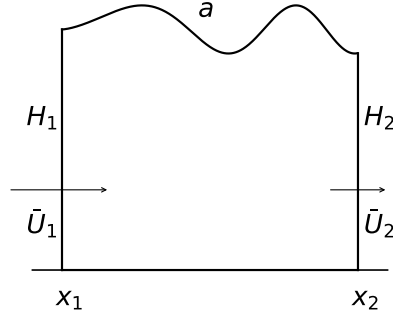


FIGURE 6. Mass continuity equation (35) can be interpreted as giving the rate of change of the time-dependent area A .

Equation (37) is an integrated form of mass continuity. To determine the derivative form, suppose the width $\Delta x = x_2 - x_1$ is small, thus $A \approx \Delta x H$. If we divide (37) by Δx and take $\Delta x \rightarrow 0$ then we get

$$\frac{\partial H}{\partial t} = a - \frac{\partial}{\partial x} (\bar{U} H) \quad (38)$$

because $\lim_{\Delta x \rightarrow 0} (\int_{x_1}^{x_2} a dx) / \Delta x = a$ is the average value of the SMB over the small interval $[x_1, x_2]$, and $\lim_{\Delta x \rightarrow 0} (\bar{U}_2 H_2 - \bar{U}_1 H_1) / \Delta x = \partial(\bar{U} H) / \partial x$ is a (partial) derivative. We have re-derived, and perhaps understood better, equation (35) in the planar case.

4. SHALLOW ICE APPROXIMATION

Shallowness of glaciers and ice sheets. Glaciers have several outstanding properties as fluid problems. As explained in Section 2, ice is slow and non-Newtonian. Also, glaciers are subject to contact slip, i.e. basal sliding, along a hard-to-observe boundary. However, it is the typical *shallowness* of glacier geometry, at least for large glaciers and ice sheets, which allows us to use a simpler stress balance model, an approximation to the full Glen-Stokes system, which enables faster simulations. Because ice sheet geometry is both shallow and slow, the problem has a distinctive *diffusive* character.

Figure 7 shows a cross-section of the Greenland ice sheet. This vertically-exaggerated view is familiar from the literature, but without such exaggeration the ice sheet is revealed to be a thin blanket, barely three-dimensional. More precisely, ice sheets are *shallow* in the sense that the dimensionless ratio $\epsilon = [H]/[L]$, between typical scales for thickness $[H]$ and horizontal (length) $[L]$, is small. For this Greenland example, $[H] = 2000$ m and $[L] = 1000$ km are representative, so $\epsilon = 0.002$.

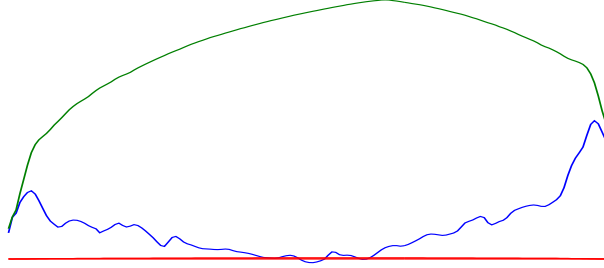


FIGURE 7. This cross-section of the Greenland ice sheet, at 71° N latitude, is 760 km wide and 3200 m thick. The slightly-thickened horizontal line (bottom; red) shows the same cross-section without vertical exaggeration.

The non-sliding, isothermal *shallow ice approximation* (SIA; [3, 12, 26]) is an effective model for shallow glaciers, as long as they are not dominated by basal sliding. Mountain glaciers have much larger ϵ ratios, but they too are often modeled within an SIA framework. A small portion of an ice sheet may have a large ϵ ratio, so one might not use the SIA. If there is substantial basal sliding, or the ice is floating, then a different shallow model, the *shallow shelf* or *membrane* approximation [26], is more appropriate. Hybrid shallow models [7], that combine two shallow approximations, are in widespread use, and comprehensive models couple these with conservation of energy [2]. However, these notes will not plunge into the thicket of modern modeling choices.

Velocity formulas in the SIA. The label “SIA” is put on several equations. First it refers to certain simplified velocity formulas. The principled way to derive these formulas starts from the full Glen-Stokes stress balance (3)–(5), and then applies a small parameter argument [11, for example], rescaling the variables using $\epsilon = [H]/[L]$, and then dropping terms which are small when ϵ is small.

However, the same destination can be reached by a lazier approach, adopted here. The SIA can simply use the slab-on-a-slope result (14) as valid even for general glaciers, not just uniform slabs. Actually, we need two assumptions for this to work: *i*) the surface s and bed b elevation functions must be smooth, and *ii*) a small-angle approximation must be adequate to replace $\sin \alpha$ in (14) with the surface slope $|\nabla s|$. Regarding *ii*), $|\sin \alpha| \approx |\tan \alpha| = |\nabla s|$ when α is small (Figure 1), and a key idea in the derivation is that the shear stresses are then linear in depth and proportional to ∇s .

For a general Glen exponent n , equation (14) becomes the following SIA formula for the horizontal velocity $\mathbf{U}(t, x, y, z)$, which is $\mathbf{U} = \langle u, v \rangle$ in components:

$$\mathbf{U} = \mathbf{U}_0 - \frac{2}{n+1} A(\rho g)^n ((s-b)^{n+1} - (s-z)^{n+1}) |\nabla s|^{n-1} \nabla s. \quad (39)$$

This formula applies on the domain (21). The first minus sign in (39) is remembered by noting that ice flows downhill, that is, opposite the ∇s direction.

When using SKE (24) to evolve geometry under the non-sliding SIA theory—see the next Section—only the surface value is needed:

$$\mathbf{U}|_s = -\frac{2}{n+1}A(\rho g)^n(s-b)^{n+1}|\nabla s|^{n-1}\nabla s. \quad (40)$$

Derive (40) by substituting $\mathbf{U}_0 = 0$ and $z = s$ into (39).

Vertical integration of (39) yields two additional formulas. First, using the power rule $\int z^{n+1} dz = z^{n+2}/(n+2) + c$, we have a formula for the ice flux:

$$\mathbf{q} = -\frac{2}{n+2}A(\rho g)^n(s-b)^{n+2}|\nabla s|^{n-1}\nabla s. \quad (41)$$

(Compare equation (34).) Second, we get the surface value of the vertical velocity:

$$\begin{aligned} w|_s &= \mathbf{U}|_s \cdot \nabla s - \nabla \cdot \mathbf{q} \\ &= -\frac{2}{n+1}A(\rho g)^n(s-b)^{n+1}|\nabla s|^{n+1} + \nabla \cdot \left(\frac{2}{n+2}A(\rho g)^n(s-b)^{n+2}|\nabla s|^{n-1}\nabla s \right). \end{aligned} \quad (42)$$

Equation (42) is derived, similar to the mass continuity equation (35), using incompressibility (3), $w|_b = 0$, and vertical integration to express w at the surface, and then the Leibniz rule (36) and $\mathbf{U}_0 = 0$. The second form in (42) follows from (40) and (41).

The formulas for $\mathbf{q}$ and $w|_s$ contain a common scalar *diffusivity* coefficient function:

$$D^{\text{SIA}} = \frac{2}{n+2}A(\rho g)^n(s-b)^{n+2}|\nabla s|^{n-1}. \quad (43)$$

This positive coefficient appears in (41) and (42), which we can now write cleanly:

$$\mathbf{q} = -D^{\text{SIA}}\nabla s. \quad (44)$$

$$w|_s = \mathbf{U}|_s \cdot \nabla s + \nabla \cdot (D^{\text{SIA}}\nabla s). \quad (45)$$

In the remainder of this section we will try to understand these SIA formulas. We will see how they can be applied in example geometries and in simulations of evolving glaciers.

Example: Computing SIA velocities for a given geometry. If the glacier geometry is known then the above formulas compute the surface velocity and the flux. We will do this approximately using finite differences (FD; Section 3), as needed for the time-dependent SIA simulations to follow, but in some cases it can be done analytically by differentiating the surface elevation function.

For planar geometry, given by known functions $b(x)$ and $s(x)$ for either steady or a snapshot in time, (40) simplifies to this formula for horizontal surface velocity:

$$u|_s = -\frac{2}{n+1}A(\rho g)^n(s-b)^{n+1} \left| \frac{\partial s}{\partial x} \right|^{n-1} \frac{\partial s}{\partial x}. \quad (46)$$

For example, suppose b is a simple incline and s is a gently-oscillating surface, as shown in Figure 8 (top). Where this surface is (nearly) flat we expect $u|_s \approx 0$, while on the steeper parts we expect significant rightward flow $u|_s > 0$, and this is what we see.

Our FD approximation of formula (46) computes the surface slope at the *staggered grid* points $x_{j+1/2}$, which are half-way between the regular grid locations $\{x_j\}$; see Section 3

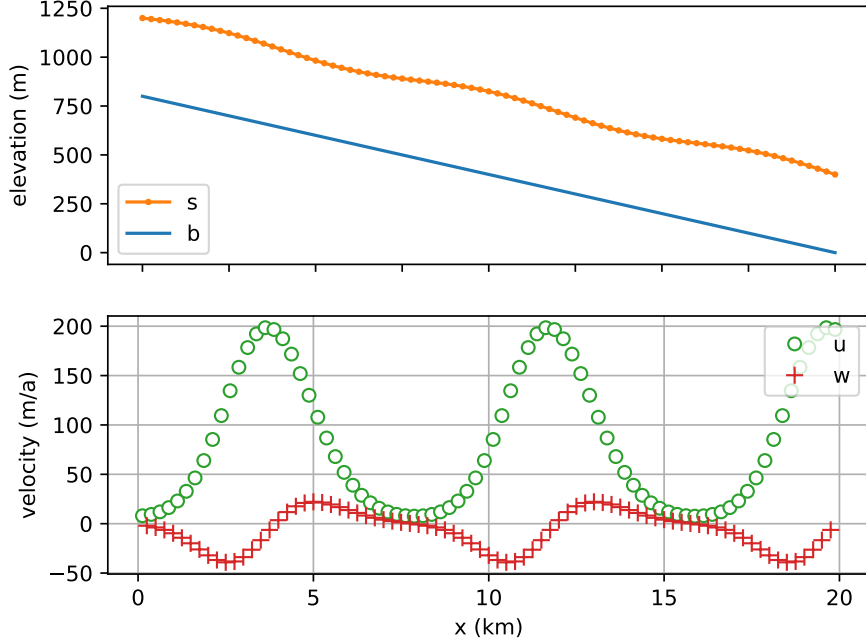


FIGURE 8. Top panel: glacier geometry. Bottom panel: FD approximations of the horizontal ($\circ$) and vertical ($+$) surface velocity in the SIA.

for grid notation. Using the staggered point achieves second-order accuracy because the function evaluations are centered around it:

$$\frac{\partial s}{\partial x}(x_{j+1/2}) = \frac{s(x_{j+1}) - s(x_j)}{\Delta x} + O((\Delta x)^2). \quad (47)$$

Averaging the ice thicknesses, symmetrically to the staggered point, is also needed, to maintain second-order accuracy. Now the approximation of (46) is

$$u_{j+1/2} = -\frac{2}{n+1} A(\rho g)^n \left(\frac{H_j + H_{j+1}}{2} \right)^{n+1} \left| \frac{s_{j+1} - s_j}{\Delta x} \right|^{n-1} \frac{s_{j+1} - s_j}{\Delta x}, \quad (48)$$

where $H_j = s_j - b_j$. Here is the corresponding Python function:

```
def u(x, b, s):
    dx = x[1] - x[0]
    dsdx = (s[1:] - s[:-1]) / dx
    Hstag = ((s[:-1] - b[:-1]) + (s[1:] - b[1:])) / 2.0
    C_u = (2.0 / (n + 1)) * A * (rhoi * g) ** n
    return -C_u * Hstag ** (n + 1) * abs(dsdx) ** (n - 1) * dsdx
```

The diffusivity D^{SIA} is computed from formula (43), again at staggered points:

$$D_{j+1/2}^{\text{SIA}} = \frac{2}{n+2} A(\rho g)^n \left(\frac{H_j + H_{j+1}}{2} \right)^{n+2} \left| \frac{s_{j+1} - s_j}{\Delta x} \right|^{n-1}. \quad (49)$$

The Python is short, and similar to the horizontal velocity:

```
def diffusiveD(x, b, s):
    dx = x[1] - x[0]
```

```

dsdx = (s[1:] - s[:-1]) / dx
Hstag = ((s[:-1] - b[:-1]) + (s[1:] - b[1:])) / 2.0
C_D = (2.0 / (n + 2)) * A * (rhoi * g) ** n
return C_D * Hstag ** (n + 2) * abs(dsdx) ** (n - 1)

```

Computing $\mathbf{q} = -D^{\text{SIA}} \frac{\partial s}{\partial x}$, if that were needed, requires only a small modification.

To compute the surface vertical velocity $w|_s$ we evaluate (45) using another centered difference, which yields values on the (interior) regular grid points. Define staggered-grid values

$$\phi_{j+1/2} = u_{j+1/2} \frac{s_{j+1} - s_j}{\Delta x} \approx \left(u|_s \frac{\partial s}{\partial x} \right)_{j+1/2}.$$

Then:

$$w_j = \frac{\phi_{j-1/2} + \phi_{j+1/2}}{2} + \frac{D_{j+1/2}^{\text{SIA}}(s_{j+1} - s_j) - D_{j-1/2}^{\text{SIA}}(s_j - s_{j-1})}{\Delta x^2}. \quad (50)$$

The short Python function which computes (50) is not shown.

Run `py/shallow.py` as follows. It calls the above functions, for a particular geometry, and it generates Figure 8:

```

$ python3 shallow.py
grid: J=80 intervals, spacing dx=250.0 m
range for u:  min =    7.6455,  max =  198.4541  (m/a)
range for w:  min =  -39.1243,  max =   22.1340  (m/a)
writing to image file output/uw.pdf

```

What does Figure 8 show? As already noted, because of gravity the flow is faster where the surface is steeper, and the horizontal velocity reflects this. The surface vertical velocity is more interesting. Through incompressibility, it causes the surface to rise in dips and fall at humps. This is a key feature of ice flow: it is *smoothing*. The vertical velocity tends to remove details of surface topography, at least at the large scales covered by the SIA. Gravity and incompressibility combine to generate ice surface motion which smoothes surfaces under SKE evolution, at least when the SMB data is also smooth (in space and time).

Figure 7 showed the result for a real ice sheet; the surface is much smoother than the base. Ice-penetrating radar in the 21st century has revealed widespread rough bedrock topography under ice sheets, making it even more obvious that ice sheet surfaces are smoother than their beds. In numerically solving the time-dependent SIA, our next task, we will also see the smoothing effect.

5. TIME-DEPENDENT GLACIER SIMULATIONS FROM THE SIA

Diffusion equations for geometry. Section 3 presented two equations which model the evolution of a glacier’s shape: the surface kinematical equation (SKE) (24) for the surface elevation s , and the mass continuity equation (35) for the thickness H . Our plan is to compute velocities and fluxes from the non-sliding shallow ice approximation (SIA; Section 4), which approximates the Glen-Stokes system of Section 2. Now these geometry evolution equations become time-dependent glacier models. Unlike the minimal SKE-based “model” in Section 3, these models are essentially self-contained.

Thus in the current Section we will combine equations from Sections 3 and 4 to build the simplest well-posed² evolution model in glaciology. Simulations built this way start from a known glacier geometry and apply a (generally) time-dependent climate, an SMB function. They generate the geometry at future times, which can be regarded as predictions.

In fact, all SIA-based geometry-evolution equations simplify to essentially the same *diffusion equation*. On the one hand, recall the SKE (24), namely

$$\frac{\partial s}{\partial t} - \mathbf{u}|_s \cdot \mathbf{n}_s = a. \quad (51)$$

Here $\mathbf{u}|_s$ is the 3D surface velocity, and recall that $\mathbf{n}_s = \left\langle -\frac{\partial s}{\partial x}, -\frac{\partial s}{\partial y}, 1 \right\rangle$ is a normal vector. From equation (45) the surface motion term has a divergence form:

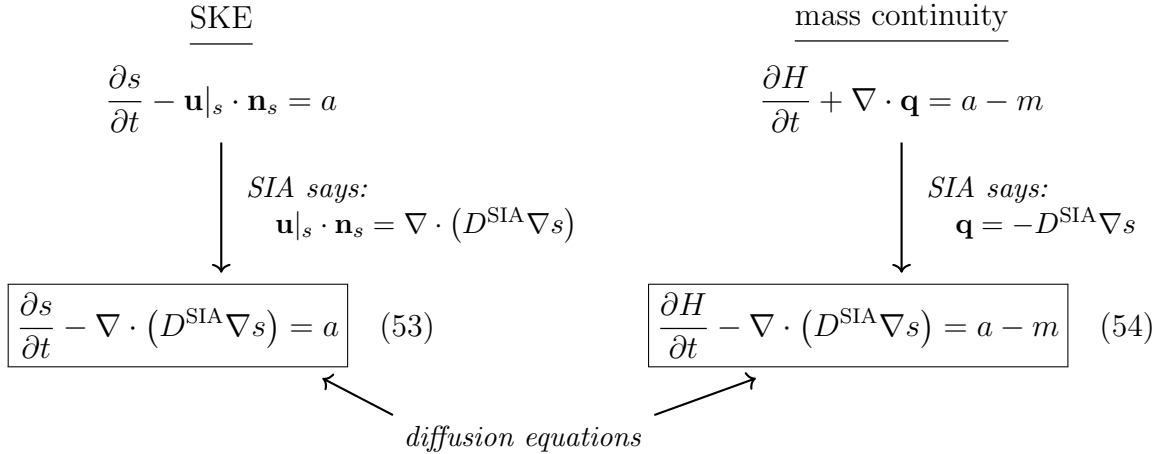
$$\mathbf{u}|_s \cdot \mathbf{n}_s = -\mathbf{U}|_s \cdot \nabla s + w|_s = \nabla \cdot (D^{\text{SIA}} \nabla s).$$

(Note that $\mathbf{U}|_s \cdot \nabla s$ is a 2D dot product.) Recall that D^{SIA} is given by equation (43), and Section 4 provides a Python function for it.

On the other hand, recall mass continuity equation (35) for the thickness $H = s - b$:

$$\frac{\partial H}{\partial t} + \nabla \cdot \mathbf{q} = a - m. \quad (52)$$

The SIA ice flux $\mathbf{q} = -D^{\text{SIA}} \nabla s$ can be substituted into this equation. Thus these (non-sliding) SIA formulas reduce to very-similar diffusion equations, the boxed equations in the following diagram.



Such similar forms are not a surprise, actually. Section 3 showed that our starting equations (51) and (52) are equivalent if we assume incompressibility (3) and a basal kinematical equation (33). Both boxed equations involve the surface slope ∇s and the ice thickness (through D^{SIA} , equation (43)). The apparent s -vs- H distinction is not that important, though the surface elevation s is a smoother geometry function in time and space than the thickness H (e.g. Figure 7).

An apparent difference between the boxed equations is that (53) does not refer to the *basal* mass balance m (Section 3). The original SKE (24) isolates the surface evolution, while the mass continuity equation (35) always involves the whole ice column. On the

²In a technical sense the well-posedness problem is only partly solved [17].

other hand, the derivation of (53) used (43), via an assumption of zero basal velocity. In the general case the basal mass balance m would reappear.

The two models (53) and (54) are similar in another respect: they must be solved subject to an inequality constraint. That is, the surface must remain above the bed ($s \geq b$) and the thickness must not be negative ($H \geq 0$).

Historically-speaking, form (54) of the mass continuity equation allowed numerical ice sheet modeling to get started. Note that diffusion equations, especially the *heat equation*, are clichéd topics in numerical analysis textbooks, for example in [19, 21]. By inventing an FD scheme for the SIA diffusivity in (54), Mary-Anne Mahaffy [20] started the field with a 1976 numerical model of the Barnes ice cap (Baffin Island, Canada).

Nonlinear and degenerate diffusivity. Equation (43) gives the diffusivity D^{SIA} , which appears in the diffusion equations (53) and (54). Recall:

$$D^{\text{SIA}} = \frac{2}{n+2} A(\rho g)^n (s-b)^{n+2} |\nabla s|^{n-1}. \quad (55)$$

Independent of the power n , the units of D^{SIA} reduce to square meters per second, as otherwise equations (53) and (54) would not be dimensionally correct. It follows that the units of the ice softness A will depend on n (Table 1).

Equation (55) expresses the dominant macroscopic effect of the Glen flow law (5), and it is at the heart of why glaciology is a (highly!) *nonlinear* science. The diffusivity is not a constant, nor a known-in-advance function, as it is in linear textbook examples [19, 21]. Instead, for $n = 3$ it scales with the 5th power of thickness and the 2nd power of surface slope. Even when $n = 1$, which is the Newtonian fluid case, diffusivity is independent of surface slope but it still depends on the 3rd power of thickness.

The local value of diffusivity D^{SIA} is the rate of smoothing of the surface elevation, over a small area, caused by ice flow. Formula (55) returns a large value when the ice is both thick and steep. Thus ice will flow downhill, away from surface maxima, and it will converge toward surface minima, most strongly at such locations.

However, it is worth noting some conceptual and practical concerns about SIA equations (53) or (54), and interpreting them as diffusion PDEs, given the nonlinearities in (55):

- i) The derivation of the SIA model is inaccurate when significant longitudinal (membrane) stresses are present. This is distinctly true when ice is floating or sliding, but also when the flow is locally confined by terrain. In such cases the SIA approximation of the true solution of the Glen-Stokes model equations (3)–(5) may be completely inadequate.
- ii) The diffusivity D^{SIA} depends on the geometry solution function, and thus PDEs (53) and (54) are nonlinear. Numerical methods are therefore more difficult to implement, and harder to analyze and debug.
- iii) The diffusivity D^{SIA} goes to zero at margins, where $H = s - b \rightarrow 0$, and at divides and domes, where $|\nabla s| \rightarrow 0$. That is, the diffusion process *degenerates* at these places. The solutions s and H of (53) or (54) are generally not smooth there, though they are continuous (if b is). Smoothness of s and H is lost at margins and domes, even when the bed b and SMB a data are smooth. The largest numerical geometry errors arise at such locations.

All of these concerns are research topics in mathematical glaciology.

Explicit scheme for the geometry SIA. Having thought about the form and meaning of these equations, we finally apply finite differences (FD) and write code.

Consider the planar case of equation (53). This is the simplest combination of the SKE and the SIA stress balance:

$$\frac{\partial s}{\partial t} - \frac{\partial}{\partial x} \left(D^{\text{SIA}} \frac{\partial s}{\partial x} \right) = a. \quad (56)$$

The simplest scheme for (56) applies forward differences in time (Euler method) and centered differences in space. We will treat the diffusivity as a coefficient which varies in time and space, evaluated on the staggered grid, $D_{j+1/2} = D^{\text{SIA}}(t_n, x_{j+1/2})$. Recall from Section 3 the notation $s_j \approx s(t_n, x_j)$ and $s_j^{\text{new}} \approx s(t_{n+1}, x_j)$, and write $a_j = a(t_n, x_j)$. Here is the scheme:

$$\frac{s_j^{\text{new}} - s_j}{\Delta t} - \frac{D_{j+1/2}(s_{j+1} - s_j)/\Delta x - D_{j-1/2}(s_j - s_{j-1})/\Delta x}{\Delta x} = a_j. \quad (57)$$

The staggered-grid diffusivities $D_{j+1/2}$ are exactly the values of D^{SIA} computed in Section 4 by `diffusiveD()`; we will re-use that Python function. The slopes $(s_{j+1} - s_j)/\Delta x$ use approximation (47). Note that our centered-difference approximation of the outer derivative in (56) is effectively differencing staggered-grid values of the flux $\mathbf{q} = -D^{\text{SIA}}\nabla s$, an idea which generalizes to two horizontal dimensions [3].

Unlike the advection problem in Section 3, a forward-time centered-space scheme (57) is acceptable for a diffusion equation like (56). At least it is *conditionally stable* [21], about which more below. Writing (57) as an update,

$$s_j^{\text{new}} = s_j + \frac{\Delta t}{\Delta x^2} [D_{j+1/2}(s_{j+1} - s_j) - D_{j-1/2}(s_j - s_{j-1})] + \Delta t a_j, \quad (58)$$

it becomes this Python in the $a = 0$ case we will be testing:

```
def geometrystep(x, s, D, dt):
    dx = x[1] - x[0]
    dsdx = (s[1:] - s[:-1]) / dx
    snew = s.copy()
    snew[1:-1] += dt * (D[1:] * dsdx[1:] - D[:-1] * dsdx[:-1]) / dx
    return snew
```

Observe that this code freezes the surface elevation at both ends, that is, `snew[0]=s[0]` and `snew[J+1]=s[J+1]`. In Section 3 we needed a single boundary condition for the SKE, which is a one-way advection in that velocity-is-given paradigm. However, in the current Section our velocities are from the SIA, and we have a second-order (in space) diffusion problem, so two Dirichlet boundary values [21] are appropriate.

Adaptive time-stepping for stability. The following time-stepping loop is *adaptive* in the sense that Δt is re-computed at each step, based on the current state of the solution:

```
t = 0
for k in range(maxsteps):
    D = diffusiveD(x, b, s)
```



```

dt = getdt(t, tf, D.max()) # adaptive time step length
if tf - t < 1.0e-6: # normal stopping criterion
    break
s = geometrystep(x, s, D, dt)
t += dt

```

The time axis is $t \in [0, t_f]$, with final time t_f ; our demonstration below uses $t_f = 40$ a. Normal termination is by the `break`, once t is within 10^{-6} seconds of the final time, to allow for rounding error, while `maxsteps` merely protects against an infinite loop.

Now, if (56) were a linear PDE, so that the staggered-grid diffusivities $D_{j+1/2}$ did not depend on the solution s , then it is well-known [19, 21] that a sufficient condition for scheme stability is

$$\frac{(\max_j D_{j+1/2}) \Delta t}{\Delta x^2} \leq C \quad \Longleftrightarrow \quad \Delta t \leq \frac{C \Delta x^2}{\max_j D_{j+1/2}} \quad (59)$$

where $C = \frac{1}{2}$. This criterion is comparable to the CFL condition (29) for the explicit upwind scheme applied to the advection problem in Section 3.

However, using condition (59) with $C = \frac{1}{2}$ in an adaptive time-stepping scheme will not necessarily give stability for a glacier simulation. PDE (56) is actually nonlinear, because of the dependence of D^{SIA} on the solution, equation (55), and a different analysis is needed. Hindmarsh's analysis in [16] concludes that $C = 1/(2n)$ is suitable in (59). Therefore we compute the time-step as follows:

```

def getdt(t, tf, maxD):
    c_D = 1.0 / 6.0 # coefficient from Hindmarsh (2001), for n=3
    dt_D = c_D * dx**2 / maxD # apply diffusive condition
    return min([dt_D, tf - t]) # do not step past end of [0,tf]

```

Example: A rough start. These pieces go into a Python program `py/dome.py` which simulates a time-dependent glacier using the SIA. The data (climate) is highly-simplified: the bed is flat ($b = 0$) and the SMB is zero ($a = 0$; also $m = 0$). Note that $H = s$ and diffusion equations (53) and (54) are identical, so the focus in this simulation is on how SIA ice flow evolves glacier shape.

Suppose we generate the initial surface elevation *randomly*. The geometry is initially horrible, compared to real glaciers. Run this experiment as follows:

```

$ python3 dome.py
grid: J=80 intervals, spacing dx=500.0 m
starting time-stepping ...
took 1814 steps to reach final time 40.0000 a
writing results to output/dome.png and output/timesteps.png

```

The evolving geometry is shown in Figure 9. Recall that the SIA formula (55), for D^{SIA} , generates large values when the ice is thick and the surface slope is steep, which certainly applies during the early part of the simulation. The geometry change rates $|\partial s / \partial t|$ are huge at the beginning. However, the ice flow is aggressively smoothing and the final surface elevation is a surprisingly symmetric and smooth dome shape, to which

we will return soon. Also note that, because of conserved mass, the margin advances as the central heights fall; the free boundary moves.

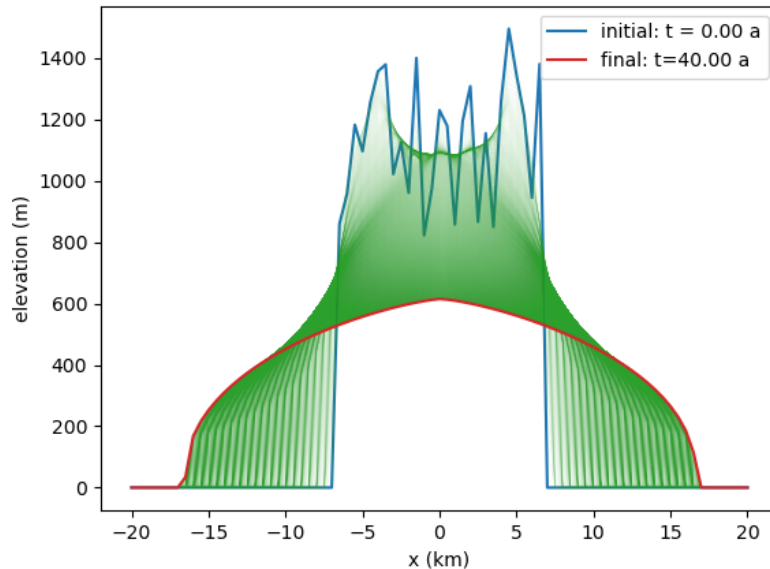


FIGURE 9. Evolution of surface elevation (and thickness) under the SIA model, from a very rough start.

The other output from the run, Figure 10, graphs the time step Δt over time. At the start the stability criteria sees enormous computed diffusivities, so by (59) the time steps are correspondingly short. Toward the end of the run the time steps stabilize, at order of a month for this resolution. Given the crazy start, this example also demonstrates the *robustness* of the adaptive time-stepping approach.

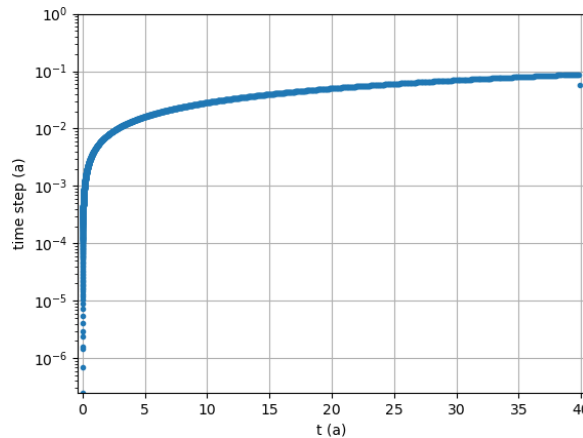


FIGURE 10. Time step length Δt , over time, from the Figure 9 simulation.

Making modifications to the program `py/dome.py` is a recommended exercise. Runs become unstable if the time step Δt is too large, although the coefficient $1/6$ can be increased quite a bit without obvious instabilities. One exercise is to throw out the adaptive time-stepping scheme entirely, but then it becomes clear why we want it. Fixing a sufficiently-short time step to permit stability will generate very long run times.

The most valuable exercise is to increase the spatial resolution, e.g. by doubling the number J of subintervals. Our program does $O(J)$ arithmetic at each time step, and of course the simulation gets more computationally expensive under spatial refinement. However, the adaptive scheme also reduces Δt so the number of time steps increases. An easy exercise shows that stability criterion (59) converts a doubling of J into an increase in arithmetic (*flops*), for a simulation to a fixed final time t_f , by a factor of eight!

Adaptive time-stepping is strongly recommended to maintain the numerical stability of glacier simulations, especially because of the strong nonlinearities in ice flow. This recommendation extends to comprehensive glacier and ice sheet models containing multiple time scales and processes [7].

Implicit schemes for glacier evolution. An alternate approach to creating stable numerical schemes is *implicitness*. For example, an implicit *backward Euler* scheme corresponding to (57) is

$$\frac{s_j^{\text{new}} - s_j}{\Delta t} - \frac{D_{j+1/2}(s_{j+1}^{\text{new}} - s_j^{\text{new}})/\Delta x - D_{j-1/2}(s_j^{\text{new}} - s_{j-1}^{\text{new}})/\Delta x}{\Delta x} = a_j. \quad (60)$$

Here the unknown surface elevations s_j^{new} appear in multiple places in the equation, so we cannot write it as an (explicit) update scheme like (58). Instead we must solve algebraic equations, simultaneously for all the values $\{s_j^{\text{new}}\}$, at each time step. Nonetheless the stability benefits of implicitness are substantial, compensating for the increased program complexity. For linear diffusions, backward Euler scheme (60) is *unconditionally stable*, meaning that arbitrarily long time steps are allowed without instability [21].

Implicitness comes with caveats. First, long time steps, i.e. large Δt values, always imply an accuracy cost, because the approximation $\partial s / \partial t \approx (s_j^{\text{new}} - s_j) / \Delta t$ becomes poor. Therefore accuracy concerns also limit the time steps, not just stability. Second, and more important, implicit schemes are more expensive in a per-time-step sense, because the unknown solution values must be treated as a vector in a (large) linear or nonlinear system of equations, which must be formed and solved at each time step.

Note that scheme (60) is really only *semi-implicit* as written. That is, it uses staggered-grid diffusivities which were evaluated, by formula (55), from the current-time surface elevations s_j . The advantage of this choice is that only a linear system needs to be solved at each time step. A *fully-implicit* scheme like the one in [3] would use the updated values s_j^{new} in the diffusivities as well, and this would require the solution of a system on nonlinear equations at each time step, for instance by Newton's method [5]. Despite the strong nonlinearities, fully-implicit SIA schemes are practical when appropriate high-performance solver tools are used.

There is a tradeoff between the easy implementability of adaptive explicit schemes and the improved stability of implicit ones. The latter have not made it into widely-used glacier models. Especially for Stokes-type models, the tradeoff depends on the poorly-understood character of geometry evolution, and the jury is still out on the ultimate best scheme [6]. Most present-day numerical glacier and ice sheet models use explicit time-stepping.

Halfar’s exact, attracting solution. The final shape in Figure 9 is a smooth and symmetrical dome, and this is no accident. It turns out that *any* flat-bed ($b = 0$) and zero SMB ($a = 0$) glacier will asymptotically approach this dome shape.

In 1981 Peter Halfar published a time-dependent SIA solution [13], and he showed that any rough surface will evolve toward this solution.³ A second paper [14] extended these results to two horizontal dimensions. The Halfar solutions are both exact and *attracting*, and thus worth knowing.

Consider the PDE you get by inserting (55) into (53). Then setting $a = b = 0$, and naming the constant $\Gamma = \frac{2}{n+2}A(\rho g)^n$, we have this simplified nonlinear diffusion equation:

$$\frac{\partial s}{\partial t} = \nabla \cdot (\Gamma s^{n+2} |\nabla s|^{n-1} \nabla s). \quad (61)$$

Halfar found the *similarity solution* of (61) by applying scaling arguments. In outline for the planar case, because $a = 0$, the ice mass is conserved so the input (x) and output (s) lengths can be re-scaled by the same unknown power of t :

$$x = t^\alpha \tilde{x}, \quad s(t, x) = t^{-\alpha} \phi(\tilde{x}) \quad (62)$$

These variables are substituted into (61). Note that the dome grows laterally (x increases) as its height falls (s decreases), which explains the signs of the powers. After much calculus one finds that t cancels from the equation if $\alpha = 1/(3n + 2)$, which is $\alpha = 1/11$ when $n = 3$. Then an exact formula for $\phi(\tilde{x})$ can be derived by solving an ordinary differential equation; this is the invariant shape of the dome. The resulting formulas appear in the program `py/dome.py`.

Figure 11 was generated by changing a boolean flag in `py/dome.py`, so that the initial condition is equal to the Halfar dome formula, evaluated at its characteristic time t_0 [8]. The simulation runs for the same duration as before, and then the program plots both the numerical and exact solutions at that time. The fit is very close.

The power $\alpha = 1/(3n + 2)$ is a manifestation of shear-thinning ice flow [22]. The Halfar solution solves the problem of putting a Dirac delta mass of ice at the origin, at some (conceptual) true starting time, and evolving its shape only through flow. The dome expands laterally at t^α rate, which is very fast when the dome is tall but becomes agonizingly slow for long times. The geometry never actually reaches steady state.

Convergence and verification. Our program `py/dome.py` calls `py/shallow.py`, so the code is already complicated. Are we sure that our implementation is correct?

These three justifications for numerical correctness appear in the modeling literature:

- i) The author made no mistakes.
- ii) The numerical results are close to those from other researchers.
- iii) The numerical results are close to an exact solution.

The first two justifications, *infallibility* and *intercomparison*, should be less common than they are. The third, preferred by the CFD community [28], is the simple idea of *verification*: the numerical error should be measured in cases where the correct answer is known. Here “correct answer” means an exact solution to the PDE itself.

³The proof, a linear asymptotic analysis, applies to any surface that is not too far from the solution. Numerical results, however, show that any large perturbation will also converge to the Halfar dome solution.

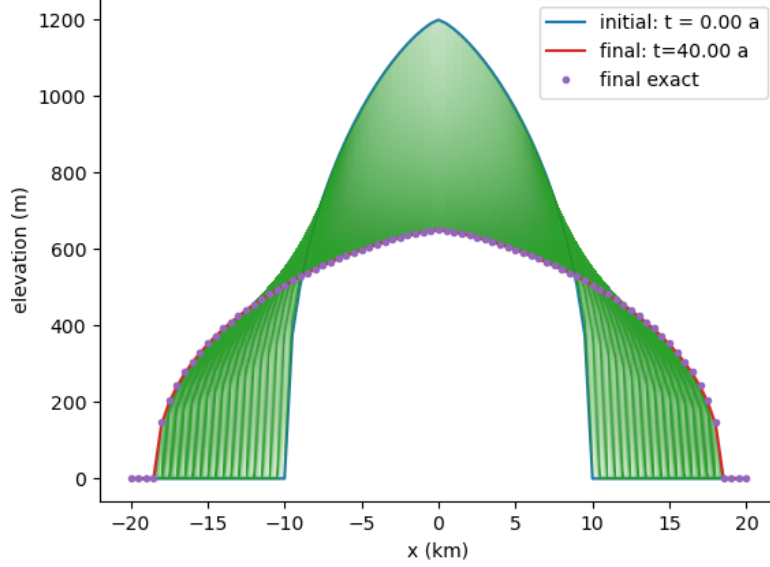


FIGURE 11. Just like Figure 9, but starting from a Halfar dome shape.

In technical terms, we intend our numerical schemes to *converge* to the exact (but usually unknown) solution of the continuum problem under *grid refinement*. Proving such convergence is a main topic in numerical textbooks [19, 21]. However, even if we expect convergence to happen in theory, we still need to *verify* it for the actual computer program. This involves measuring the accuracy of the scheme by finding its empirical rate of convergence in particular cases. Almost any implementation mistake will break convergence, or drastically reduce its rate; the fragility of the verification process is its strongest feature.

In theory, for a linear diffusion equation in the form of (56), the scheme (58) makes $O(\Delta t) + O((\Delta x)^2)$ numerical errors. These *local truncation error* [21] rates come merely from examining the FD formulas. They are the best we can expect when measuring the *numerical error* compared to the exact solution.

However, the convergence rate for glacier equations is reduced because of the singular-gradient margin shape. For an experiment, run `py/dome.py` with resolutions $J = 40, 80, 160, 320, 640$. The measured errors against the Halfar solution (Figure 11) are as follows:

J	Δx (m)	maximum (m)	average (m)
40	1000	66.52	6.94
80	500	7.24	1.79
160	250	8.57	0.77
320	125	15.63	0.36
640	62.5	9.42	0.12

The two columns are the maximum and average numerical elevation errors at the final time. The average error decreases with improved grid resolution, as expected for a correctly-implemented code, at a measured rate $O((\Delta x)^{1.40})$. By contrast, because the numerical error in horizontal margin location will be comparable to one grid space, and because the gradient there is unbounded, the maximum error behaves erratically [8].

The Halfar solution is simple to use for verification, but such elegant exact solutions are rare. However, exact solutions for verification can actually be constructed at will, by the *method of manufactured solutions* [28]. The idea is to choose a desired solution, then compute analytically what source term would make it satisfy the PDE exactly, then run the numerical scheme with that source term, and finally compare against the chosen solution. Almost any coding mistake will corrupt the convergence rate.

If in physics there's something you don't understand, you can always hide behind the uncharted depths of nature. You can always blame God. You didn't make it so complex yourself. But if your program doesn't work, there is no one to hide behind. You cannot hide behind obstinate nature. If it doesn't work, you've messed up. E. Dijkstra

REFERENCES

- [1] D. J. ACHESON, *Elementary Fluid Dynamics*, Oxford University Press, 1990.
- [2] A. ASCHWANDEN, E. BUELER, C. KHROULEV, AND H. BLATTER, *An enthalpy formulation for glaciers and ice sheets*, Journal of Glaciology, 58 (2012), pp. 441–457.
- [3] E. BUELER, *Stable finite volume element schemes for the shallow ice approximation*, Journal of Glaciology, 62 (2016), pp. 230–242.
- [4] ———, *Conservation laws for free-boundary fluid layers*, SIAM J. Appl. Math., 81 (2021), pp. 2007–2032.
- [5] ———, *PETSc for Partial Differential Equations: Numerical Solutions in C and Python*, SIAM Press, Philadelphia, 2021.
- [6] ———, *Performance analysis of high-resolution ice-sheet simulations*, J. Glaciol., 69 (2023), pp. 930–935.
- [7] E. BUELER AND J. BROWN, *Shallow shelf approximation as a “sliding law” in a thermomechanically coupled ice sheet model*, Journal of Geophysical Research: Earth Surface, 114 (2009).
- [8] E. BUELER, C. S. LINGLE, J. A. KALLEN-BROWN, D. N. COVEY, AND L. N. BOWMAN, *Exact solutions and verification of numerical models for isothermal ice sheets*, J. Glaciol., 51 (2005), pp. 291–306.
- [9] J. COGLEY, R. HOCK, L. RASMUSSEN, A. ARENDT, A. BAUDER, R. B. WAITE, P. JANSSON, G. KASER, M. MÖLLER, L. NICHOLSON, AND M. ZEMP, *Glossary of glacier mass balance and related terms*, IHP-VII Technical Documents in Hydrology 86, UNESCO-IHP, Paris, 2011.
- [10] R. COURANT, K. FRIEDRICHS, AND H. LEWY, *Über die partiellen differenzengleichungen der mathematischen physik*, Mathematische Annalen, 100 (1928), pp. 32–74.
- [11] A. C. FOWLER, *Mathematical Models in the Applied Sciences*, Cambridge Univ. Press, 1997.
- [12] R. GREVE AND H. BLATTER, *Dynamics of Ice Sheets and Glaciers*, Springer, Berlin, 2009.
- [13] P. HALFAR, *On the dynamics of the ice sheets*, Journal of Geophysical Research: Oceans, 86 (1981), pp. 11065–11072.
- [14] ———, *On the dynamics of the ice sheets 2*, Journal of Geophysical Research: Oceans, 88 (1983), pp. 6043–6051.
- [15] C. R. HARRIS, K. J. MILLMAN, S. J. VAN DER WALT, R. GOMMERS, ET AL., *Array programming with NumPy*, Nature, 585 (2020), pp. 357–362.
- [16] R. C. A. HINDMARSH, *Notes on basic glaciological computational methods and algorithms*, in Continuum Mechanics and Applications in Geophysics and the Environment, Springer, 2001, pp. 222–249.
- [17] G. JOUVET AND E. BUELER, *Steady, shallow ice sheets as obstacle problems: well-posedness and finite element approximation*, SIAM J. Appl. Math., 72 (2012), pp. 1292–1314.
- [18] G. JOUVET AND J. RAPPAZ, *Analysis and finite element approximation of a nonlinear stationary Stokes problem arising in glaciology*, Advances in Numerical Analysis, (2011).

- [19] R. J. LEVEQUE, *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*, SIAM Press, 2007.
- [20] M. W. MAHAFFY, *A three-dimensional numerical model of ice sheets: tests on the Barnes Ice Cap, Northwest Territories*, J. Geophys. Res., 81 (1976), pp. 1059–1066.
- [21] K. W. MORTON AND D. F. MAYERS, *Numerical Solution of Partial Differential Equations: An Introduction*, Cambridge University Press, 2nd ed., 2005.
- [22] J. F. NYE, W. B. DURHAM, P. M. SCHENK, AND J. M. MOORE, *The instability of a south polar cap on Mars composed of carbon dioxide*, Icarus, 144 (2000), pp. 449–455.
- [23] F. PATTYN, *A new three-dimensional higher-order thermomechanical ice sheet model: Basic sensitivity, ice stream development, and ice flow across subglacial lakes*, Journal of Geophysical Research: Solid Earth, 108 (2003).
- [24] F. RATHGEBER, D. A. HAM, L. MITCHELL, M. LANGE, F. LUPORINI, A. T. T. McRAE, G.-T. BERCEA, G. R. MARKALL, AND P. H. J. KELLY, *Firedrake: Automating the finite element method by composing abstractions*, ACM Trans. Math. Softw., 43 (2016), pp. 24:1–24:27.
- [25] J. C. SAVAGE AND W. S. B. PATERSON, *Borehole measurements in the Athabasca Glacier*, Journal of Geophysical Research, 68 (1963), pp. 4521–4536.
- [26] C. SCHOOF AND I. J. HEWITT, *Ice-sheet dynamics*, Annual Review of Fluid Mechanics, 45 (2013), pp. 217–239.
- [27] C. J. VAN DER VEEN, *Fundamentals of Glacier Dynamics*, CRC Press, 2nd ed., 2013.
- [28] P. WESSELING, *Principles of Computational Fluid Dynamics*, Springer, 2001.